

CO4 ASSESSMENT TOOL 2(Industrial Problem Solving)

Name: Y.Thanushma

Registration Number:192425355

Course Code:CSA0617-Design and Analysis of Algorithms

1. Cybersecurity Access Control Optimization (Search + Graph Algorithms)

Introduction

In a corporate network, access control ensures that users can access only the resources they are authorized to use. As organizations grow, managing permissions manually becomes difficult. Graph algorithms provide an efficient way to model and optimize access relationships between users, roles, and resources.

Modeling Access Control Using Graphs

A graph consists of nodes and edges.

- **Nodes:** Users, Roles, Resources, and Permissions.
- **Edges:** Access relationships between these entities.

Example:

- User A $\rightarrow$ Manager Role
- Manager Role $\rightarrow$ Financial Database
- Manager Role $\rightarrow$ HR Records

This graph structure helps visualize and analyze access paths efficiently.

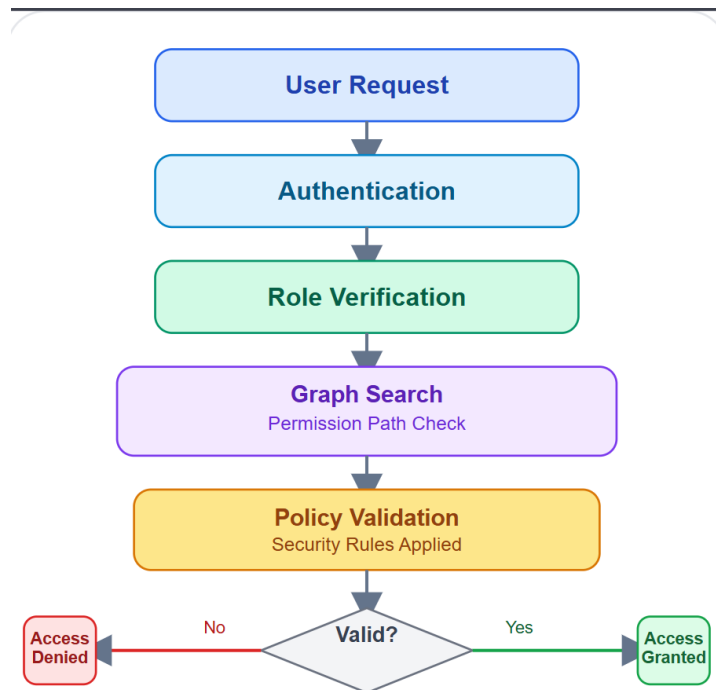
Constraints in Access Control

The system must satisfy several security constraints:

1. **Role-Based Access Control (RBAC):** Users receive permissions through assigned roles.

2. **Least Privilege Principle:** Users obtain only the permissions required for their tasks.
3. **Separation of Duties:** Critical operations require multiple authorized roles.
4. **Policy Compliance:** Access decisions must follow organizational security policies.
5. **Dynamic User Management:** Permissions should update automatically when roles change.

Optimized Access Control System Design



Scalability in Large Organizations

Large enterprises may contain:

- Thousands of users
- Hundreds of roles
- Millions of access relationships

Scalability can be improved through:

- Graph databases
- Distributed processing
- Role hierarchies

- Incremental graph updates

These techniques reduce computation time and storage requirements.

Computational Performance Evaluation

Algorithm	Purpose	Time Complexity
BFS	Access verification	$O(V + E)$
DFS	Permission analysis	$O(V + E)$
Dijkstra	Shortest access path	$O(E \log V)$
Cycle Detection	Privilege escalation detection	$O(V + E)$

Where:

- V = Number of vertices (users, roles, resources)
- E = Number of edges (access relationships)

Graph-based systems provide faster permission analysis compared to manual access control methods.

Results and Interpretation

The graph-based access control system:

- Improves security by identifying unauthorized access paths.
- Reduces administrative effort through automated role management.
- Supports rapid permission verification.
- Scales effectively for large organizations.
- Enhances compliance with security policies.

Conclusion

Graph algorithms provide an efficient solution for cybersecurity access control optimization. By modeling users, roles, and resources as graph structures, organizations can enforce security policies, detect access anomalies, and manage permissions efficiently. The approach offers high scalability, better performance, and stronger security for modern enterprise networks.

2.Smart Grid Fault Detection Optimization

Introduction:

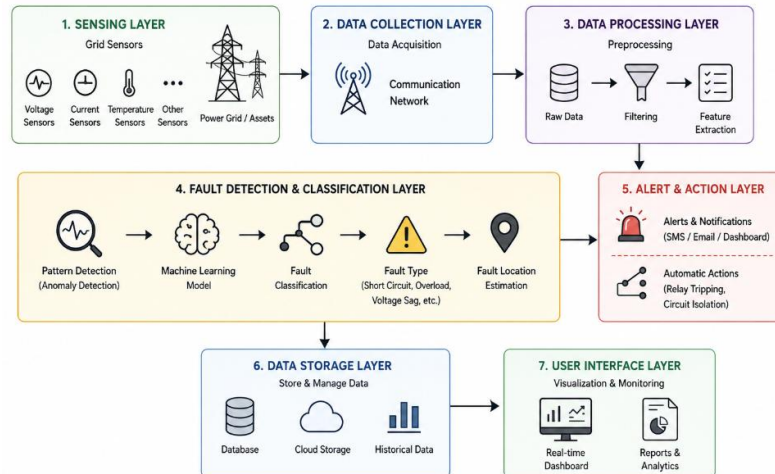
A smart grid uses sensors to continuously monitor voltage, current, frequency, and power. Pattern detection algorithms identify abnormal sensor patterns and detect faults quickly.

Constraints:

- Real-time processing: Faults must be detected quickly.
- Data accuracy: Sensor errors can cause false alarms.
- Large data volume: Thousands of sensors generate continuous data.
- Low latency: Detection should happen with minimum delay.

Proposed System:

Sensors → Data Collection → Preprocessing → Pattern Detection → Fault Classification → Alert



The system analyzes sensor readings and detects patterns such as sudden voltage drops or current increases. Threshold methods can detect immediate faults, while machine-learning algorithms can classify complex faults.

Scalability:

Large networks can be managed using edge computing, cloud processing, distributed systems, and parallel processing. This reduces processing and communication delays.

Performance Evaluation:

The system can be evaluated using:

- Accuracy
- Precision and Recall
- False Alarm Rate
- Detection Time

A good system should provide high accuracy, low false alarms, and fast detection.

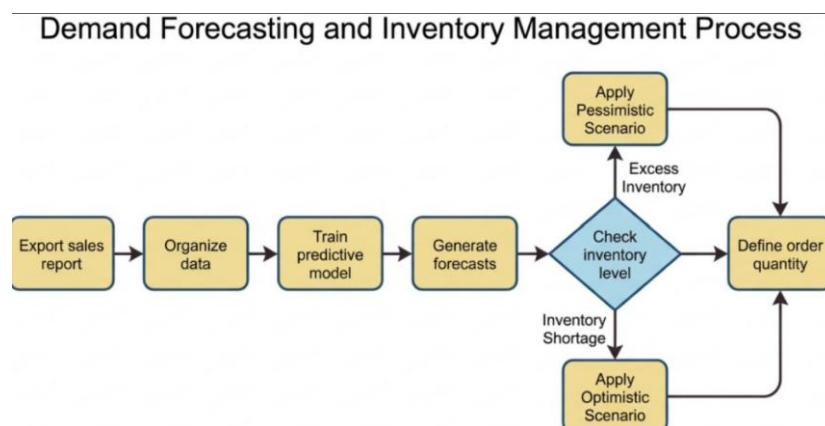
Conclusion:

Pattern detection enables quick and reliable smart-grid fault detection. It reduces outage duration, protects equipment, and improves overall power-system reliability and stability.

3.Retail Demand Forecasting Optimization (Dynamic Programming)

Introduction

Retail companies need accurate demand forecasts to maintain proper inventory levels. Dynamic Programming (DP) divides the forecasting problem into smaller stages and finds the best inventory decision for each period



Constraints

- Seasonal trends: Demand changes during festivals and holidays.
- Data variability: Customer demand may change unexpectedly.
- Inventory limits: Storage capacity and budget are limited.
- Forecast accuracy: Incorrect predictions can cause overstock or shortages.

Proposed Solution

Historical Data → Data Preprocessing → Demand Prediction → DP Optimization → Inventory Decision

The system analyzes past sales and seasonal patterns. DP evaluates different inventory decisions and selects the solution that minimizes ordering, storage, and shortage costs.

Implementation Challenges

- Handling large amounts of sales data.
- Adapting to sudden demand changes.
- Updating forecasts frequently.
- Managing computational resources.

Computational Complexity

For n time periods and m inventory states, DP generally requires $O(n \times m)$ time and $O(m)$ memory.

Results

The optimized forecast helps maintain proper inventory levels, reduces storage and shortage costs, and improves product availability.

Conclusion

Dynamic Programming provides an efficient approach for demand forecasting and inventory optimization. It helps retailers make better stocking decisions and improves supply-chain efficiency and profitability.

4.Smart Home Energy Management Optimization (Greedy + DP)

Introduction

A smart home contains appliances that consume electricity at different times. Optimization algorithms can schedule appliances efficiently to reduce electricity cost while maintaining user comfort.

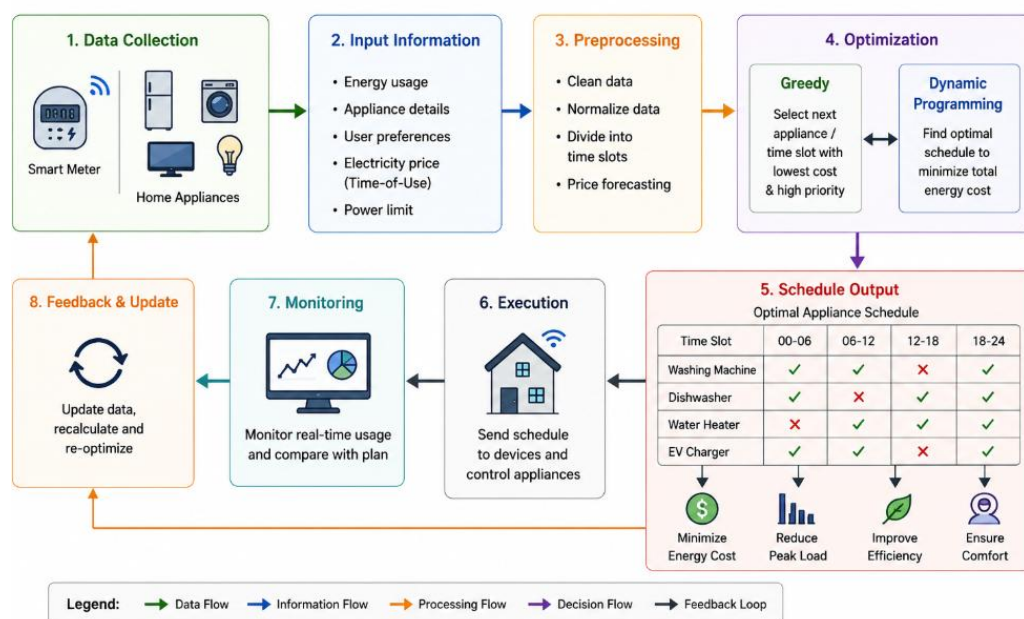
Constraints

- Energy pricing: Electricity prices may vary by time.
- User preferences: Some appliances must operate at specific times.
- Power limits: Total energy usage should remain within the available limit.
- Device priorities: Important appliances should receive higher priority.

Proposed Solution

Smart Meter → Energy Monitoring → Optimization → Appliance Scheduling → Energy Saving

The Greedy algorithm can schedule appliances with the lowest-cost or highest-priority options first. Dynamic Programming (DP) can evaluate different combinations of appliance schedules and select the combination with minimum total energy cost.



Scalability

For multiple devices, the system can use automated scheduling, grouped appliance management, and efficient data processing. This allows the system to handle many smart appliances.

Performance

- Greedy: Fast and simple, approximately $O(n \log n)$ when sorting is required.
- Dynamic Programming: Provides better optimization but requires more computation.
- Performance can be measured using energy cost, energy consumption, execution time, and user comfort.

Results

The optimized schedule can shift flexible appliances to cheaper time periods, reduce peak energy consumption, and lower electricity bills.

Conclusion

Combining Greedy and Dynamic Programming provides an efficient smart-home energy management solution. Greedy offers fast scheduling, while DP provides better overall optimization, resulting in lower energy costs and improved energy efficiency.